



Linux
Professional
Institute

101.3 Runlevel wechseln und das System anhalten oder neu starten

Referenz zu den LPI-Lernzielen

LPIC-1 v5, Exam 101, Objective 101.3

Gewichtung

3

Hauptwissensgebiete

- das Standard-Runlevel oder das Boot-Target setzen
- zwischen Runleveln und Boot-Targets wechseln, einschließlich des Single-User-Modus
- Systemhalt und Neustart von der Befehlszeile aus
- Benutzer vor einem Runlevel- bzw. Boot-Target-Wechsel oder einem anderen größeren Ereignis benachrichtigen
- Prozesse korrekt beenden
- Kenntnis von acpid

Auszugsweise Liste der verwendeten Dateien, Begriffe und Hilfsprogramme

- `/etc/inittab`
- `shutdown`
- `init`
- `/etc/init.d/`
- `telinit`
- `systemd`
- `systemctl`
- `/etc/systemd/`

- `/usr/lib/systemd/`
- `wall`



101.3 Lektion 1

Zertifikat:	LPIC-1
Version:	5.0
Thema:	101 Systemarchitektur
Lernziel:	101.3 101.3 Runlevel wechseln und das System anhalten oder neu starten
Lektion:	1 von 1

Einführung

Ein gemeinsames Merkmal aller Betriebssysteme, die der Unix-Philosophie folgen, ist der Einsatz separater Prozesse zur Steuerung verschiedener Funktionen des Systems. Diese Prozesse, *Daemons* genannt (oder allgemeiner *Dienste*), sind auch für erweiterte Funktionen verantwortlich, die dem Betriebssystem zugrunde liegen, wie z.B. Netzwerkanwendungsdienste (HTTP-Server, Dateifreigabe, E-Mail usw.), Datenbanken, bedarfsgerechter Konfiguration usw. Obwohl Linux einen monolithischen Kernel verwendet, werden viele Aspekte des Betriebssystems auf niedriger Ebene durch Daemons beeinflusst, wie z.B. Lastausgleich und Firewall-Konfiguration.

Welche Daemons aktiv sein sollten, hängt vom Zweck des Systems ab. Die Menge der aktiven Daemons sollte auch zur Laufzeit veränderbar sein, so dass Dienste gestartet und gestoppt werden können, ohne das gesamte System neu starten zu müssen. Um dieses Problem zu lösen, bietet jede größere Linux-Distribution eine Art Dienstverwaltungsprogramm zur Steuerung des Systems an.

Dienste können durch Shell-Skripte oder durch ein Programm und deren unterstützenden Konfigurationsdateien gesteuert werden. Die erste Methode wird durch den *SysVinit*-Standard implementiert, der auch als *System V* oder nur *SysV* bekannt ist. Die zweite Methode wird durch

systemd und *Upstart* implementiert. Historisch gesehen wurden SysV-basierte Servicemanager am häufigsten von Linux-Distributionen verwendet. Heute sind systembasierte Servicemanager in den meisten Linux-Distributionen häufiger zu finden. Der Servicemanager ist das erste Programm, welches vom Kernel während des Bootprozesses gestartet wird, daher ist seine PID (Prozess-Identifikationsnummer) immer 1.

SysVinit

Ein auf dem SysVinit-Standard basierender Servicemanager stellt vordefinierte Sätze von Systemzuständen, *runlevels* genannt, und die entsprechenden auszuführenden Serviceskriptdateien zur Verfügung. Runlevels sind von 0 bis 6 nummeriert und werden im Allgemeinen den folgenden Zwecken zugeordnet:

Runlevel 0

Systemabschaltung.

Runlevel 1, s oder single

Einzelbenutzermodus, ohne Netzwerk und andere nicht wesentliche Funktionen (Wartungsmodus).

Runlevel 2, 3 oder 4

Mehrbenutzer-Modus. Benutzer können sich per Konsole oder Netzwerk anmelden. Die Runlevel 2 und 4 werden nicht häufig verwendet.

Runlevel 5

Mehrbenutzer-Modus. Entspricht 3, inklusive der Anmeldung im grafischen Modus.

Runlevel 6

Systemneustart.

Das Programm, das für die Verwaltung der Runlevel und der zugehörigen Daemons/Ressourcen verantwortlich ist, heißt */sbin/init*. Während der Systeminitialisierung identifiziert das Programm *init* den angeforderten Runlevel, der durch einen Kernelparameter oder in der Datei */etc/inittab* definiert ist, und lädt die zugehörigen Skripte, die dort für den gegebenen Runlevel aufgelistet sind. Jeder Runlevel kann viele zugehörige Servicedateien haben, üblicherweise Skripte im Verzeichnis */etc/init.d/*. Da nicht alle Runlevel in den verschiedenen Linux-Distributionen gleichwertig sind, kann eine kurze Beschreibung des Zwecks des Runlevels auch in SysV-basierten Distributionen gefunden werden.

Die Syntax der */etc/inittab*-Datei verwendet folgendes Format:

```
id:runlevels:action:process
```

Die `id` ist ein generischer Name der zur Identifizierung eines Eintrags verwendet wird und kann bis zu vier Zeichen lang sein. Der Eintrag `runlevels` ist eine Liste von Runlevelnummern, für die eine bestimmte Aktion ausgeführt werden soll. Der Begriff `action` definiert, wie `init` den durch den Begriff `process` bezeichneten Prozess ausführt. Die verfügbaren Aktionen lauten:

boot

Der Prozess wird während der Systeminitialisierung ausgeführt. Das Feld `runlevels` wird ignoriert.

bootwait

Der Prozess wird während der Systeminitialisierung ausgeführt und `init` wartet, bis er beendet ist, um fortzufahren. Das Feld `runlevels` wird ignoriert.

sysinit

Der Prozess wird nach der Systeminitialisierung ausgeführt, unabhängig von der Ausführungsebene. Das Feld `runlevels` wird ignoriert.

wait

Der Prozess wird für die gegebenen Run-Levels ausgeführt und `init` wartet, bis er beendet ist, um fortzufahren.

respawn

Der Prozess wird neu gestartet, wenn er abgebrochen wird.

ctrlaltdel

Der Prozess wird ausgeführt, wenn der Init-Prozess das `SIGINT`-Signal empfängt, welches ausgelöst wird, wenn die Tastaturkombination `Strg` + `Alt` + `Entf` gedrückt wird.

Der Standard-Runlevel — derjenige, der gewählt wird, wenn kein anderer als Kernelparameter angegeben wird — wird ebenfalls in `/etc/inittab`, im Eintrag `id:x:initdefault` definiert. Das `x` steht für die Nummer des Standard-Runlevels. Diese Nummer sollte niemals `0` oder `6` sein, da sie das System zum Herunterfahren oder Neustarten veranlassen würde, sobald es den Bootprozess beendet hat. Eine typische `/etc/inittab`-Datei ist hier zu sehen:

```
# Default runlevel
id:3:initdefault:

# Configuration script executed during boot
```

```

si::sysinit:/etc/init.d/rcS

# Action taken on runlevel S (single user)
~:S:wait:/sbin/sulogin

# Configuration for each execution level
10:0:wait:/etc/init.d/rc 0
11:1:wait:/etc/init.d/rc 1
12:2:wait:/etc/init.d/rc 2
13:3:wait:/etc/init.d/rc 3
14:4:wait:/etc/init.d/rc 4
15:5:wait:/etc/init.d/rc 5
16:6:wait:/etc/init.d/rc 6

# Action taken upon ctrl+alt+del keystroke
ca::ctrlaltdel:/sbin/shutdown -r now

# Enable consoles for runlevels 2 and 3
1:23:respawn:/sbin/getty tty1 VC linux
2:23:respawn:/sbin/getty tty2 VC linux
3:23:respawn:/sbin/getty tty3 VC linux
4:23:respawn:/sbin/getty tty4 VC linux

# For runlevel 3, also enable serial
# terminals ttyS0 and ttyS1 (modem) consoles
S0:3:respawn:/sbin/getty -L 9600 ttyS0 vt320
S1:3:respawn:/sbin/mgetty -x0 -D ttyS1

```

Der Befehl `telinit q` sollte jedes Mal ausgeführt werden, nachdem die Datei `/etc/inittab` modifiziert wurde. Das Argument `q` (oder `Q`) weist `init` an, seine Konfiguration neu zu laden. Ein solcher Schritt ist wichtig, um einen Systemstopp aufgrund einer falschen Konfiguration in `/etc/inittab` zu vermeiden.

Die Skripte, die von `init` zur Einrichtung der einzelnen Runlevel verwendet werden, sind im Verzeichnis `/etc/init.d/` gespeichert. Jeder Runlevel hat ein zugehöriges Verzeichnis in `/etc/`, namens `/etc/rc0.d/`, `/etc/rc1.d/`, `/etc/rc2.d/`, usw., mit den Skripten, die beim Start des entsprechenden Runlevels ausgeführt werden sollen. Da dasselbe Skript von verschiedenen Runleveln benutzt werden kann, sind die Dateien in diesen Verzeichnissen nur symbolische Links zu den eigentlichen Skripten in `/etc/init.d/`. Außerdem gibt der erste Buchstabe des Linknamens im Verzeichnis des Runlevels an, ob der Dienst für den entsprechenden Runlevel gestartet oder beendet werden soll. Ein Linkname, der mit dem Buchstaben `K` beginnt, legt fest, dass der Dienst beim Betreten des Runlevels beendet wird (kill). Ein Linkname, der mit dem

Buchstaben **S** beginnt, legt fest, dass der Dienst beim Betreten des Runlevels gestartet wird (**start**). Das Verzeichnis `/etc/rc1.d/` beinhaltet beispielsweise viele Links zu Netzwerkskripten, die mit dem Buchstaben **K** beginnen, da der Runlevel 1 der Runlevel für einen einzelnen Benutzer ohne Netzwerkverbindung darstellt.

Der Befehl **runlevel** zeigt den aktuellen Runlevel für das System an. Der Befehl **runlevel** zeigt dabei zwei Werte an. Der Erste ist der vorherige Runlevel und der Zweite der aktuelle Runlevel:

```
$ runlevel
N 3
```

Der Buchstabe **N** in der Ausgabe zeigt an, dass sich der Runlevel seit dem letzten Bootvorgang nicht geändert hat. Im Beispiel ist der **runlevel 3** der aktuelle Runlevel des Systems.

Dasselbe **init** Programm kann verwendet werden, um zwischen den Runleveln in einem laufenden System zu wechseln, ohne dass ein Neustart erforderlich ist. Der Befehl **telinit** kann ebenfalls verwendet werden, um zwischen den Runleveln zu wechseln. Zum Beispiel wechseln die Befehle **telinit 1**, **telinit s** oder **telinit S** das System auf Runlevel 1.

systemd

Gegenwärtig ist **systemd** der am weitesten verbreitete Werkzeugsatz zur Verwaltung von Systemressourcen und -diensten, die von **systemd** als *Units* bezeichnet werden. Eine Unit besteht aus einem Namen, einem Typ und einer entsprechenden Konfigurationsdatei. Zum Beispiel wird eine Unit für einen *httpd* Server-Prozess (wie der Apache-Webserver) bei Red Hat-basierten Distributionen **httpd.service** genannt und seine Konfigurationsdatei wird ebenfalls **httpd.service** genannt (bei Debian-basierten Distributionen wird diese Unit **apache2.service** genannt).

Es gibt sieben verschiedene Arten von Systemunits:

service

Der gebräuchlichste Unit-Typ, für aktive Systemressourcen, die initiiert, unterbrochen und wieder geladen werden können.

socket

Der Socket Unit-Typ kann ein Dateisystemsocket oder ein Netzwerksocket sein. Alle Socket Units verfügen über eine entsprechende Serviceunit, die geladen wird, wenn der Socket eine Anfrage erhält.

device

Eine Device-Unit ist einem Hardwaregerät zugeordnet, welche durch den Kernel identifiziert wird. Ein Gerät wird nur dann als Systemunit betrachtet, wenn eine udev-Regel für diesen Zweck existiert. Eine Geräteunit kann verwendet werden, um Konfigurationsabhängigkeiten aufzulösen, wenn bestimmte Hardware erkannt wird, vorausgesetzt, dass Eigenschaften aus der udev-Regel als Parameter für die Geräteeinheit verwendet werden können.

mount

Eine Mount-Unit ist eine Einhängpunktdefinition im Dateisystem, ähnlich wie ein Eintrag in `/etc/fstab`.

automount

Eine Automount-Unit ist ebenfalls eine Einhängpunktdefinition im Dateisystem, wird aber automatisch eingehängt. Jede Automount-Unit hat eine entsprechende Mount-Unit, die beim Zugriff auf den Automount-Einhängpunkt initiiert wird.

target

Eine Target-Unit ist eine Gruppierung von anderen Units, die als eine einzige Unit verwaltet wird.

snapshot

Eine Snapshot-Unit ist ein gespeicherter Zustand des Systemmanagers (nicht auf jeder Linux-Distribution verfügbar).

Der Hauptbefehl zur Steuerung von Systemunits ist `systemctl`. Der Befehl `systemctl` wird zur Ausführung aller Aufgaben bezüglich der Aktivierung, Deaktivierung, Ausführung, Unterbrechung, Überwachung usw. von Units verwendet. Bei einer fiktiven Unit mit dem Namen `unit.service` werden beispielsweise die häufigsten `systemctl` Aktionen wie folgt lauten:

systemctl start unit.service

Startet `unit`.

systemctl stop unit.service

Stoppt `unit`.

systemctl restart unit.service

Startet `unit` neu.

systemctl status unit.service

Zeigt den Zustand von `unit` an, einschließlich ob dieser läuft oder nicht.

systemctl is-active unit.service

Zeigt *active* an, wenn *unit* läuft oder *inactive* wenn nicht.

systemctl enable unit.service

Aktiviert *unit*, d.h. *unit* wird während der Systeminitialisierung geladen.

systemctl disable unit.service

unit wird nicht mit dem System gestartet.

systemctl is-enabled unit.service

Überprüft, ob *unit* mit dem System gestartet wird. Die Antwort wird in der Variable *\$?* gespeichert. Der Wert *0* zeigt an, dass *unit* mit dem System startet, anderenfalls wird der Wert *1* gesetzt.

NOTE

Bei neueren Installationen von *systemd* wird die Konfiguration einer Unit zum Bootzeitpunkt dargestellt. Zum Beispiel:

```
$ systemctl is-enabled apparmor.service
enabled
```

Wenn keine anderen Units mit dem gleichen Namen im System vorhanden sind, kann das Suffix nach dem Punkt entfallen. Gibt es z.B. nur eine *httpd*-Unit vom Typ *service*, dann genügt *httpd* als Unitparameter für *systemctl*.

Der Befehl *systemctl* kann auch *Systemziele* steuern. Die Unit *multi-user.target* kombiniert zum Beispiel alle Units, die von der Mehrbenutzersystemumgebung benötigt werden. Sie ähnelt dem Runlevelnummer 3 in einem System, das SysV verwendet.

Der Befehl *systemctl isolate* wechselt zwischen verschiedenen Zielen. Um manuell zum Ziel *multi-user* zu wechseln, benutzt man:

```
# systemctl isolate multi-user.target
```

Es gibt entsprechende Ziele zu SysV-Runleveln, beginnend mit *runlevel0.target* bis hin zu *runlevel6.target*. Allerdings benutzt *systemd* nicht die Datei */etc/inittab*. Um das Standard-Systemziel zu ändern, kann die Option *systemd.unit* zur Kernelparameterliste hinzugefügt werden. Um zum Beispiel *multi-user.target* als Standardziel zu verwenden, sollte der Kernelparameter *systemd.unit=multi-user.target* lauten. Alle Kernelparameter können durch Ändern der Bootloaderkonfiguration persistent gemacht werden.

Eine andere Möglichkeit, das Standardziel zu ändern, besteht darin, den symbolischen Link `/etc/systemd/system/default.target` so zu modifizieren, dass dieser auf das gewünschte Ziel verweist. Die Neudefinition des Links kann mit dem Befehl `systemctl` selbst vorgenommen werden:

```
# systemctl set-default multi-user.target
```

Ebenso kann mit dem folgenden Befehl überprüft werden, was das Standard-Bootziel des Systems ist:

```
$ systemctl get-default  
graphical.target
```

Ähnlich wie bei Systemen, die SysV verwenden, sollte das Standardziel niemals auf `shutdown.target` zeigen, da es dem Runlevel 0 (Herunterfahren) entspricht.

Die zu jeder Unit gehörenden Konfigurationsdateien befinden sich im Verzeichnis `/lib/systemd/system/`. Der Befehl `systemctl list-unit-files` listet alle verfügbaren Units auf und zeigt an, ob sie beim Hochfahren des Systems startfähig sind. Die Option `--type` wählt nur die Units für einen bestimmten Typ aus, wie in `systemctl list-unit-files --type=service` und `systemctl list-unit-files --type=target`.

Aktive Units oder Units, die während der aktuellen Systemsitzung aktiv waren, können mit dem Befehl `systemctl list-units` aufgelistet werden. Wie die Option `list-unit-files` wählt der Befehl `systemctl list-units --type=service` nur Units vom Typ `service` und der Befehl `systemctl list-units --type=target` nur Units vom Typ `target` aus.

Systemd ist auch für das Auslösen von leistungsbezogenen Ereignissen, sowie das Reagieren darauf, verantwortlich. Der Befehl `systemctl suspend` versetzt das System in einen Stromsparmmodus und hält die aktuellen Daten im Speicher. Der Befehl `systemctl hibernate` kopiert alle Speicherdaten auf die Festplatte, so dass der aktuelle Zustand des Systems nach dem Ausschalten wiederhergestellt werden kann. Die mit solchen Ereignissen verbundenen Aktionen werden in der Datei `/etc/systemd/logind.conf` oder in separaten Dateien innerhalb des Verzeichnisses `/etc/systemd/logind.conf.d/` definiert. Diese systemd-Funktion kann jedoch nur genutzt werden, wenn kein anderer Powermanager im System läuft, wie z.B. der `acpid`-Daemon. Der `acpid`-Daemon ist der Hauptpowermanager für Linux und erlaubt feinere Anpassungen der Aktionen nach leistungsbezogenen Ereignissen, wie Schließen des Laptopdeckels, niedrigem Batterie- oder Akkuladezustand.

Upstart

Die von Upstart verwendeten Initialisierungsskripte befinden sich im Verzeichnis `/etc/init/`. Systemdienste können mit dem Befehl `initctl list` aufgelistet werden, der auch den aktuellen Status der Dienste und, falls verfügbar, ihre PID-Nummer anzeigt.

```
# initctl list
avahi-cups-reload stop/waiting
avahi-daemon start/running, process 1123
mountall-net stop/waiting
mountnfs-bootclean.sh start/running
nmbd start/running, process 3085
passwd stop/waiting
rc stop/waiting
rsyslog start/running, process 1095
tty4 start/running, process 1761
udev start/running, process 1073
upstart-udev-bridge start/running, process 1066
console-setup stop/waiting
irqbalance start/running, process 1842
plymouth-log stop/waiting
smbd start/running, process 1457
tty5 start/running, process 1764
failsafe stop/waiting
```

Jede Aktion von Upstart hat ihren eigenen unabhängigen Befehl. Zum Beispiel kann der Befehl `start` verwendet werden, um ein sechstes virtuelles Terminal zu initiieren:

```
# start tty6
```

Der aktuelle Status einer Ressource kann mit dem Befehl `status` überprüft werden:

```
# status tty6
tty6 start/running, process 3282
```

Und die Unterbrechung eines Dienstes erfolgt mit dem Befehl `stop`:

```
# stop tty6
```

Upstart benutzt nicht die Datei `/etc/inittab`, um Runlevel zu definieren, aber die älteren

Befehle `runlevel` und `telinit` können immer noch benutzt werden, um Runlevel zu verifizieren und zwischen Runleveln zu wechseln.

NOTE

Upstart wurde für die Ubuntu Linux-Distribution entwickelt, um das parallele Starten von Prozessen zu erleichtern. Ubuntu hat die Verwendung von Upstart seit 2015 eingestellt, als es von Upstart zu `systemd` wechselte.

Herunterfahren und Neustart

Ein sehr traditioneller Befehl, der zum Herunterfahren oder Neustarten des Systems verwendet wird, wird wenig überraschend `shutdown` genannt. Der Befehl `shutdown` fügt dem Ausschaltvorgang zusätzliche Funktionen hinzu: Er benachrichtigt automatisch alle angemeldeten Benutzer mit einer Warnmeldung in ihren Shellsitzungen und verhindert neue Anmeldungen. Der Befehl `shutdown` fungiert als Vermittler zu SysV- oder Systemprozeduren, d.h. er führt die angeforderte Aktion aus, indem er die entsprechende Aktion in dem vom System angenommenen Dienstmanager aufruft.

Nach der Ausführung von `shutdown` empfangen alle Prozesse das Signal `SIGTERM`, gefolgt vom Signal `SIGKILL`, dann fährt das System herunter oder ändert seinen Runlevel. Wenn weder die Optionen `-h` noch `-r` verwendet werden, wechselt das System standardmäßig zur Ausführungsebene 1, d.h. zum Einzelbenutzermodus. Um die voreingestellten Optionen von `shutdown` zu ändern, sollte der Befehl mit folgender Syntax ausgeführt werden:

```
$ shutdown [OPTIONEN] ZEIT [NACHRICHT]
```

Nur der Parameter `ZEIT` ist erforderlich. Der Parameter `ZEIT` definiert, wann die angeforderte Aktion ausgeführt wird, wobei folgende Formate akzeptiert werden:

hh:mm

Dieses Format gibt die Ausführungszeit in Stunden und Minuten an.

+m

Dieses Format gibt an, wie viele Minuten vor der Ausführung gewartet werden soll.

now oder +0

Dieses Format bestimmt die sofortige Ausführung.

Der Parameter `NACHRICHT` ist der Warntext, der an alle Terminalsitzungen eingeloggter Benutzer gesendet wird.

Die SysV-Implementierung ermöglicht die Einschränkung der Benutzer, die in der Lage sind, den Rechner durch Drücken von `Strg` + `Alt` + `Entf` neu zu starten. Dies ist möglich, indem man die Option `-a` für den Befehl `shutdown` in die Zeile bezüglich `ctrlaltdel` in der Datei `/etc/inittab` einfügt. Auf diese Weise können nur Benutzer, deren Benutzername in der Datei `/etc/shutdown.allow` steht, das System mit der Tastenkombination `Strg` + `Alt` + `Entf` neu starten.

Der Befehl `systemctl` kann in Systemen, die `systemd` verwenden, auch zum Ausschalten oder Neustarten der Maschine verwendet werden. Um das System neu zu starten, sollte der Befehl `systemctl reboot` verwendet werden. Um das System abzuschalten, sollte der Befehl `systemctl poweroff` verwendet werden. Beide Befehle erfordern root-Rechte, um ausgeführt werden zu können, da normale Benutzer derartige Prozeduren nicht ausführen dürfen.

Einige Linux-Distributionen verknüpfen `poweroff` und `reboot` mit `systemctl` als einzelne Befehle. Zum Beispiel:

NOTE

```
$ sudo which poweroff
/usr/sbin/poweroff
$ sudo ls -l /usr/sbin/poweroff
lrwxrwxrwx 1 root root 14 Aug 20 07:50 /usr/sbin/poweroff -> /bin/systemctl
```

Nicht alle Wartungsaktivitäten erfordern ein Abschalten oder einen Neustart des Systems. Wenn es jedoch notwendig ist, den Zustand des Systems in den Einzelbenutzermodus zu ändern, ist es wichtig, angemeldete Benutzer zu warnen, damit diese nicht durch ein abruptes Ende ihrer Aktivitäten beeinträchtigt werden.

Ähnlich wie der Befehl `shutdown` beim Ausschalten oder Neustart des Systems, ist der Befehl `wall` in der Lage, eine Nachricht an Terminalsitzungen aller angemeldeten Benutzer zu senden. Dazu muss der Systemadministrator lediglich eine Datei bereitstellen oder die Nachricht direkt als Parameter an den Befehl `wall` übergeben.

Geführte Übungen

1. Wie könnte der Befehl `telinit` verwendet werden, um das System neu zu starten?

2. Was passiert mit den Diensten, die sich auf die Datei `/etc/rc1.d/K90network` beziehen, wenn das System Runlevel 1 aktiviert?

3. Wie könnte ein Benutzer mit dem Befehl `systemctl` überprüfen, ob die Unit `sshd.service` läuft?

4. Basierend auf der Nutzung von `systemd`: Welcher Befehl muss ausgeführt werden, um die Aktivierung von `sshd.service` während der Systeminitialisierung zu ermöglichen?

Offene Übungen

1. In einem SysV-basierten System wird angenommen, dass der in `/etc/inittab` definierte Standard-Runlevel 3 ist, das System aber immer im Runlevel 1 startet. Was ist die wahrscheinliche Ursache dafür?
2. Obwohl die Datei `/sbin/init` in systemd-basierten Systemen gefunden werden kann, ist sie nur ein symbolischer Link zu einer anderen ausführbaren Datei. Worauf zeigt in solchen Systemen die Datei mit `/sbin/init`?
3. Wie kann das Standardsystemziel in einem systemd-basierten System verifiziert werden?
4. Wie kann ein mit dem Befehl `shutdown` eingeplanter Systemneustart abgebrochen werden?

Zusammenfassung

Diese Lektion behandelt die wichtigsten Dienstprogramme, die von Linux-Distributionen als Servicemanager verwendet werden. Die Dienstprogramme SysVinit, systemd und Upstart haben jeweils ihren eigenen Ansatz zur Steuerung von Systemdiensten und Systemzuständen. Die Lektion geht dabei auf folgende Themen ein:

- Was Systemdienste sind und ihre Rolle im Betriebssystem.
- Konzepte und grundlegende Verwendung von SysVinit, systemd und Upstart-Befehlen.
- Wie man Systemdienste und das System selbst ordnungsgemäß startet, stoppt und neu startet.

Die behandelten Befehle und Verfahren lauten:

- Befehle und Dateien im Zusammenhang mit SysVinit, wie `init`, `/etc/inittab` und `telinit`.
- Der Hauptbefehl von systemd: `systemctl`.
- Upstart-Befehle: `initctl`, `status`, `start`, `stop`.
- Traditionelle Befehle zur Energieverwaltung, wie `shutdown`, und dem Befehl `wall`.

Lösungen zu den geführten Übungen

1. Wie könnte der Befehl `telinit` verwendet werden, um das System neu zu starten?

Der Befehl `telinit 6` wechselt zu Runlevel 6, d.h. das System wird neu gestartet.

2. Was passiert mit den Diensten, die sich auf die Datei `/etc/rc1.d/K90network` beziehen, wenn das System Runlevel 1 aktiviert?

Aufgrund des Buchstabens `K` am Anfang des Dateinamens werden die entsprechenden Dienste beendet.

3. Wie könnte ein Benutzer mit dem Befehl `systemctl` überprüfen, ob die Unit `sshd.service` läuft?

Mit dem Befehl `systemctl status sshd.service` oder `systemctl is-active sshd.service`.

4. Basierend auf der Nutzung von `systemd`: Welcher Befehl muss ausgeführt werden, um die Aktivierung von `sshd.service` während der Systeminitialisierung zu ermöglichen?

Der Befehl `systemctl enable sshd.service` wird von root ausgeführt.

Lösungen zu den offenen Übungen

1. In einem SysV-basierten System wird angenommen, dass der in `/etc/inittab` definierte Standard-Runlevel 3 ist, das System aber immer im Runlevel 1 startet. Was ist die wahrscheinliche Ursache dafür?

Die Parameter 1 oder S können in der Parameterliste des Kernels vorhanden sein.

2. Obwohl die Datei `/sbin/init` in systemd-basierten Systemen gefunden werden kann, ist sie nur ein symbolischer Link zu einer anderen ausführbaren Datei. Worauf zeigt in solchen Systemen die Datei mit `/sbin/init`?

Die Hauptsystembinärdatei: `/lib/systemd/systemd`.

3. Wie kann das Standardsystemziel in einem systemd-basierten System verifiziert werden?

Der symbolische Link `/etc/systemd/system/default.target` verweist auf die Unit-Datei, die als Standardziel definiert ist. Der Befehl `systemctl get-default` kann ebenfalls verwendet werden.

4. Wie kann ein mit dem Befehl `shutdown` eingeplanter Systemneustart abgebrochen werden?

Der Befehl `shutdown -c` sollte verwendet werden.